

Warm up

I observe the following interaction in a Linux terminal:

```
$ pwd
```

```
/home/cos217/student
```

```
$ ls
```

```
assignment0 assignment1 projects readme.txt
```

- How do I change into the projects directory?
- How do I return to the student directory from projects?
- What is the absolute path of the readme.txt file?

Precept 2: More Linux and Some Emacs

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

January 29, 2026

Overview

- 1 Navigating the Linux command line
- 2 Using Emacs
- 3 Summary

Commonly-used commands

- Directory-related commands

- `cd`: change directory
- `ls`: list files in a directory
- `pwd`: print working directory
- `mkdir`: make a new directory
- `rmdir`: remove an *empty* directory

- File-related commands

- `cp`: copy files
- `mv`: move or rename files
- `rm`: remove files
- `touch`: create an empty file or update file timestamp
- `cat`: concatenate and display file contents
- `more`: view file contents one page at a time
- `less`: less is more
- `most`: most is less and more?

- See *The Linux Operating System and the Bash Shell* handout for a more comprehensive list

Data streams and redirection

- Every process has three standard data streams:
 - Standard input (stdin): file descriptor 0
 - Standard output (stdout): file descriptor 1
 - Standard error (stderr): file descriptor 2
- By default, stdin is connected to the keyboard, stdout and stderr are connected to the terminal display
- You can redirect these streams using operators:
 - >: redirect stdout to a file (overwrite)
 - >>: redirect stdout to a file (append)
 - 2>: redirect stderr to a file
 - &>: redirect both stdout and stderr to a file
 - <: redirect stdin from a file
- Can combine them: `./prog < in.txt > out.txt 2> error.log`
- Or, direct stdout and stderr to same file: `./prog &> all.log`

Other useful bash features

- Command history: use the up and down arrow keys to navigate through previously entered commands
 - Use `Ctrl + R` to search through command history
 - `history` command to view a list of past commands (this list is a file that you can edit!)
- Tab completion: use Tab to auto-complete file and directory names
- Piping: use the pipe operator `|` to send the output of one command as input to another command
 - Example: `ls -l | less` views a long list of files one page at a time
- Aborting a process: press `Ctrl + C` to terminate a running command
 - This sends a `SIGINT` signal to the process
 - Useful for stopping programs that are taking too long or stuck in an infinite loop

The .bashrc and .bash_profile files

- Located in your home directory: `/home/uname/.bashrc`
- Customise your shell environment by adding commands to this file
- Common customisations:
 - Setting environment variables (e.g., `PATH`)
 - Creating aliases (e.g., `alias ll='ls -al'`)
 - Customising the prompt string (e.g., `PS1='[\h:\w]\$ '`)
- Force changes to take effect by running `source ~/.bashrc`

What, then, is .bash_profile?

- .bash_profile is a script that runs automatically when you start a bash shell
- Also located in your home directory: `/home/uname/.bash_profile`
- Common practice to source .bashrc from within .bash_profile

Overview

- 1 Navigating the Linux command line
- 2 Using Emacs
- 3 Summary

Why bother with Emacs?

- Versatile and extensible editor
- Integrated with gcc and gdb → useful for C programming
- Highly customisable and widely available on Unix-like systems
- Terminal-based editor → can be used over pure SSH connections
 - Can work on your phone!

Commonly-used Emacs commands + Walkthrough

- C-`o`: hold down Control and press the key `o`
- M-`o`: press Esc, release, then press the key `o`
- Navigation
 - C-a: move to the beginning of the line
 - C-e: move to the end of the line
 - C-x 1 <n>: move to line number `n`
- Editing
 - C-k: cut (kill) text from the cursor to the end of the line
 - C-y: paste (yank) previously cut text
 - C-_: undo previous change
- Saving and exiting
 - C-x C-s: save the current file
 - C-x C-c: save file and exit Emacs
- Running a function: M-x <function-name>
- See *Emacs Tutorial and Reference* handout for more

Overview

- 1 Navigating the Linux command line
- 2 Using Emacs
- 3 Summary

Summary

- More about navigating the Linux command line: creating and removing files and directories, viewing file contents, redirecting input and output, `.bashrc` and `.bash_profile`
- How and why to use Emacs
- Dates to remember:
 - Assignment 0: due Tue, 2/3 at 9pm
 - Assignment 1: due Tue, 2/10 at 9pm